

F1 Lap Time Simulator: a quasi-static point-mass approach

1. Executive summary & project overview

This project develops a Quasi-Static Lap Time Simulator (LTS) programmed in MATLAB. The core objective is to simulate and analyze the performance limits of a Formula 1 car around mathematically approximated tracks (such as Monza and Monaco). By applying fundamental principles of vehicle dynamics, fluid mechanics, and tire friction limits, the simulator generates a comprehensive telemetry profile including velocity envelopes, longitudinal forces, and a lateral/longitudinal acceleration footprint (G-G Diagram). Instead of treating vehicle speed shifts as instantaneous, this simulation models the vehicle with a finite powertrain capability (~1000 hp) and realistic aerodynamic constraints, providing a clean, high-fidelity look into track-specific engineering challenges.

```
%% LAP TIME SIMULATOR
clear; clc; close all;

%% 1. TRACK SELECTION MENU
track_list = {'Synthetic Test Track', 'Monza GP', 'Monaco GP'};
track_choice = menu('Select the track for the simulation:', track_list);
if track_choice == 0; track_choice = 1; end

%% 2. VEHICLE PARAMETERS
m = 800;           % Mass [kg]
g = 9.81;          % Gravity [m/s^2]
rho = 1.225;       % Air density [kg/m^3]
Cl = 3.5;          % Downforce coefficient
Cd = 1.2;          % Drag coefficient
A = 1.6;           % Frontal area [m^2]
mu = 1.6;          % Tire friction coefficient

% --- LONGITUDINAL PARAMETERS ---
P_engine = 745000; % Engine Power (~1000 hp) [W]
max_brake_g = 4.5; % Max braking deceleration [g]
v_min_allowed = 10; % Minimum speed to avoid division by zero [m/s]

%% 3. TRACK GEOMETRY DEFINITION
dx = 5; % Spatial step size [m]
switch track_choice
    case 1 % Synthetic Test Track
        distance = 0:dx:1000;
```

```

curvature = zeros(size(distance));
curvature(distance >= 400 & distance <= 600) = 1/40;    % Right turn
curvature(distance >= 800 & distance <= 950) = -1/150; % Left turn
track_name = 'Synthetic Test Track';

case 2 % Monza GP (Signed Geometric Approximation)
    distance = 0:dx:5793;
    curvature = zeros(size(distance));
    % Prima Variante (Variante del Rettifilo): Right then Left
    curvature(distance >= 1100 & distance <= 1140) = 1/20;
    curvature(distance >= 1150 & distance <= 1200) = -1/20;
    % Variante della Roggia: Left then Right
    curvature(distance >= 1900 & distance <= 1960) = -1/40;
    curvature(distance >= 1970 & distance <= 2040) = 1/40;
    % Lesmo 1 & Lesmo 2: Both Right turns
    curvature(distance >= 2550 & distance <= 2680) = 1/100;
    curvature(distance >= 2730 & distance <= 2850) = 1/90;
    % Variante Ascari: Left, Right, Left
    curvature(distance >= 4150 & distance <= 4220) = -1/120;
    curvature(distance >= 4230 & distance <= 4300) = 1/100;
    curvature(distance >= 4310 & distance <= 4400) = -1/130;
    % Parabolica (Curva Alboreto): Long Right turn
    curvature(distance >= 5250 & distance <= 5600) = 1/140;
    track_name = 'Monza GP';

case 3 % Monaco GP (Signed Geometric Approximation)
    distance = 0:dx:3337;
    curvature = zeros(size(distance));
    curvature(distance >= 400 & distance <= 480) = 1/35;    % Sainte
Devote (Right)
    curvature(distance >= 1150 & distance <= 1230) = -1/30; % Mirabeau
Haute (Left)
    curvature(distance >= 1250 & distance <= 1330) = 1/15; % Grand
Hotel Hairpin (Right Hairpin)
    curvature(distance >= 1340 & distance <= 1400) = -1/20; % Mirabeau
Bas (Left)
    curvature(distance >= 1550 & distance <= 1700) = 1/25; % Portier
(Right)
    curvature(distance >= 2250 & distance <= 2350) = -1/35; % Nouvelle
Chicane (Left)
    curvature(distance >= 2360 & distance <= 2450) = 1/35; % Nouvelle
Chicane (Right)
    curvature(distance >= 2750 & distance <= 2900) = 1/25; % Rascasse
(Right)
    curvature(distance >= 2950 & distance <= 3050) = 1/20; % Anthony
Noghes (Right)
    track_name = 'Monaco GP';
end
N = length(distance);

```

2. Aerodynamic and tire friction modeling

The mathematical core of the lateral performance depends heavily on aerodynamic downforce, which fundamentally alters the normal load on the tires as a function of speed. The maximum cornering velocity (v_{limit}) at any given point of signed curvature ($\kappa = 1/R$) is derived from the lateral force equilibrium. The maximum available tire grip force must balance the centrifugal force:

$$F_{lateral} = \mu \cdot F_{vertical}$$

Substituting the aerodynamic downforce equation into the vertical load, where

$$F_{vertical} = m \cdot g + F_{downforce} :$$

$$m \cdot \kappa \cdot v^2 = \mu \cdot \left(m \cdot g + \frac{1}{2} \rho C_l A v^2 \right)$$

By isolating velocity (v), we establish the absolute cornering speed threshold dictated by the tire-grip envelope:

$$v_{limit} = \sqrt{\frac{\mu \cdot m \cdot g}{m \cdot \kappa - \mu \cdot \frac{1}{2} \rho C_l A}}$$

If the denominator becomes zero or negative, it indicates that aerodynamic downforce theoretically yields infinite lateral stability relative to the track's curvature profile, in which case the vehicle performance defaults to its maximum mechanical top speed.

```
%% 4. CORNERING SPEED LIMIT CALCULATION (Grip Limited)
v_limit = zeros(1, N);
for i = 1:N
    C = curvature(i);
    if C == 0
        v_limit(i) = 350 / 3.6; % Top speed straight [m/s]
    else
        num = mu * m * g;
        den = (m * C) - (mu * 0.5 * rho * Cl * A);
        if den > 0
            v_limit(i) = min(sqrt(num / den), 350/3.6);
        else
            v_limit(i) = 300 / 3.6;
        end
    end
end
end
```

3. Longitudinal dynamics via forward/backward integration

To calculate how the vehicle transitions between straight-line speed and tight corner apexes, a dual-pass numerical integration algorithm is implemented:

3.1 Forward pass (acceleration limited)

Starting from the first spatial node, the simulation integrates forward in space (dx). The engine force is power-limited ($F_{engine} = P/v$), competing directly against atmospheric drag:

$$a_{accel} = \frac{F_{engine} - F_{drag}}{m} = \frac{\frac{P}{v} - \frac{1}{2}\rho C_d A v^2}{m}$$

Using the kinematic step integration, the maximum achievable next-step speed is limited by the upcoming corner apex speed:

$$v_{k+1} = \sqrt{v_k^2 + 2 \cdot a_{accel} \cdot dx}$$

3.2 Backward pass (braking limited)

To simulate aggressive, realistic braking points, the algorithm executes a second integration pass in reverse, working backwards from the apex of each corner. Formula 1 braking is modeled via deceleration bounds helped by aero drag:

$$a_{decel} = \frac{F_{brake} + F_{drag}}{m} = \frac{(max_brake_g \cdot m \cdot g) + \frac{1}{2}\rho C_d A v^2}{m}$$

$$v_{k-1} = \sqrt{v_k^2 + 2 \cdot a_{decel} \cdot dx}$$

3.3 Velocity envelope synthesis

The final velocity profile ($v_{simulated}$) is computed by taking the mathematical lower bound (minimum envelope) of the forward acceleration pass, the backward braking pass, and the absolute tire lateral cornering limit at every individual spatial step:

$$v_{simulated} = \min(v_{forward}, v_{backward}, v_{limit})$$

```
%% 5. LONGITUDINAL DYNAMICS
```

```
% --- FORWARD PASS (ACCELERATION) ---
```

```
v_forward = zeros(1, N);
```

```

v_forward(1) = v_limit(1); % Start speed

for i = 1:(N-1)
    v_current = max(v_forward(i), v_min_allowed);

    F_aerodrag = 0.5 * rho * Cd * A * v_current^2;
    F_engine = P_engine / v_current;
    F_net = F_engine - F_aerodrag;

    accel = F_net / m;

    % Kinematic equation: v_next^2 = v_current^2 + 2*a*dx
    v_next = sqrt(max(v_current^2 + 2 * accel * dx, v_min_allowed^2));

    % Cannot exceed the cornering limit of the next point
    v_forward(i+1) = min(v_next, v_limit(i+1));
end

% --- BACKWARD PASS (BRAKING) ---
v_backward = zeros(1, N);
v_backward(end) = v_limit(end); % End speed

for i = N:-1:2
    v_current = max(v_backward(i), v_min_allowed);

    % Total braking force (Mechanical Brake + Aero Drag helping)
    F_aerodrag = 0.5 * rho * Cd * A * v_current^2;
    F_brake = max_brake_g * g * m;
    F_decel_total = F_brake + F_aerodrag;

    decel = F_decel_total / m; % Positive value for physics equation

    % Kinematic equation backwards
    v_prev = sqrt(max(v_current^2 + 2 * decel * dx, v_min_allowed^2));

    v_backward(i-1) = min(v_prev, v_limit(i-1));
end

% --- FINAL VELOCITY PROFILE (minimum envelope) ---
v_simulated = min(v_forward, v_backward);

%% 6. LAP TIME CALCULATION
lap_time = sum(dx ./ v_simulated);

fprintf('\n--- SIMULATION RESULTS FOR: %s ---\n', upper(track_name));

```

```

--- SIMULATION RESULTS FOR: MONZA GP ---

```

```

fprintf('Estimated Lap Time: %.3f seconds\n', lap_time);

```

Estimated Lap Time: 76.929 seconds

4. Engineering telemetry insights & G-G diagram analysis

- **The G-G Footprint:** The combined acceleration diagram successfully captures the decoupled boundaries of the model. The vertical column indicates pure longitudinal actions (straight-line acceleration and heavy braking spikes up to -5 g). The horizontal points trace pure cornering efforts where traction or braking is balanced to maximize lateral force generation.
- **Track Layout Integration:** The 2D track reconstruction seamlessly visualizes the speed distribution via a thermal heatmap, emphasizing heavy deceleration zones into tight sections and rapid acceleration transitions on long straights.

```
%% 7. PLOTTING RESULTS
% 7.1 Reconstruction of 2D Track Geometry using Signed Curvature
x = zeros(1, N);
y = zeros(1, N);
heading = 0;

for i = 1:(N-1)
    heading = heading + curvature(i) * dx;
    x(i+1) = x(i) + dx * cos(heading);
    y(i+1) = y(i) + dx * sin(heading);
end

% 7.2 Advanced Telemetry Calculations (G-G Diagram)
ax_m_s2 = diff(v_simulated.^2) / (2 * dx);
ax_m_s2 = [ax_m_s2, ax_m_s2(end)];
ax_g = ax_m_s2 / g;

% Lateral Accel (a_y = v^2 * kappa) converted in G forces
ay_m_s2 = (v_simulated.^2) .* curvature;
ay_g = ay_m_s2 / g;

% --- MAIN TELEMETRY FIGURE ---
fig1 = figure('Color', [0.08 0.09 0.12], 'Name', ['F1 Telemetry - '
track_name], 'Position', [100, 100, 1300, 720]);

%% Subplot 1: Speed Profile vs Distance
ax1 = subplot(3, 3, [1, 2, 4, 5]);
set(ax1, 'Color', [0.12 0.13 0.17], 'XColor', [0.8 0.8 0.8], 'YColor', [0.0
0.9 1.0], 'GridColor', [0.3 0.3 0.3]);
hold(ax1, 'on'); grid(ax1, 'on');

yyaxis left
```

```

plot(distance, v_simulated * 3.6, 'Color', [0.0 0.9 1.0], 'LineWidth', 2.5,
'DisplayName', 'V Simulated');
plot(distance, v_limit * 3.6, 'Color', [0.5 0.5 0.6], 'LineWidth', 1.2,
'LineStyle', '--', 'DisplayName', 'V GG-Limit');
ylabel('Vehicle Speed [km/h]', 'Color', [0.0 0.9 1.0], 'FontWeight', 'bold');
ylim([0, 370]);

yyaxis right
plot(distance, abs(curvature), 'Color', [1.0 0.3 0.3], 'LineWidth', 1.5,
'DisplayName', 'Track Curvature');
ylabel('Curvature [1/m]', 'Color', [1.0 0.3 0.3], 'FontWeight', 'bold');
ax1.YAxis(2).Color = [1.0 0.3 0.3];

title(['LAP TELEMETRY ANALYSIS: ' upper(track_name)], 'Color', 'w',
'FontSize', 12, 'FontWeight', 'bold');
xlabel('Distance [m]', 'Color', [0.8 0.8 0.8]);
legend('Location', 'southwest', 'TextColor', 'w', 'EdgeColor', [0.3 0.3
0.3], 'Color', [0.12 0.13 0.17]);

%% Subplot 2: Longitudinal Accel vs Distance
ax2 = subplot(3, 3, [7, 8]);
set(ax2, 'Color', [0.12 0.13 0.17], 'XColor', [0.8 0.8 0.8], 'YColor', [0.8
0.8 0.8], 'GridColor', [0.3 0.3 0.3]);
hold(ax2, 'on'); grid(ax2, 'on');
plot(distance, ax_g, 'Color', [1.0 0.8 0.0], 'LineWidth', 1.8);
plot(distance, zeros(size(distance)), 'Color', [0.5 0.5 0.5], 'LineStyle',
'--');
ylabel('Longitudinal Accel. [G]', 'FontWeight', 'bold');
xlabel('Distance [m]');
ylim([-max_brake_g-0.5, 1.5]);

%% Subplot 3: 2D Track Layout Heatmap
ax3 = subplot(3, 3, [3, 6]);
set(ax3, 'Color', [0.08 0.09 0.12], 'XColor', 'none', 'YColor', 'none');
hold(ax3, 'on');
scatter(ax3, x, y, 25, v_simulated * 3.6, 'filled');
colormap(ax3, jet);
cb = colorbar('southoutside', 'Color', 'w');
cb.Label.String = 'Speed [km/h]';
cb.Label.Color = 'w';
axis(ax3, 'equal');
title('2D TRACK LAYOUT MAP', 'Color', 'w', 'FontSize', 10, 'FontWeight',
'bold');

%% Subplot 4: G-G Diagram
ax4 = subplot(3, 3, 9);
set(ax4, 'Color', [0.12 0.13 0.17], 'XColor', [0.8 0.8 0.8], 'YColor', [0.8
0.8 0.8], 'GridColor', [0.3 0.3 0.3]);
hold(ax4, 'on'); grid(ax4, 'on');

```

```

scatter(ax4, ay_g, ax_g, 15, v_simulated * 3.6, 'filled');

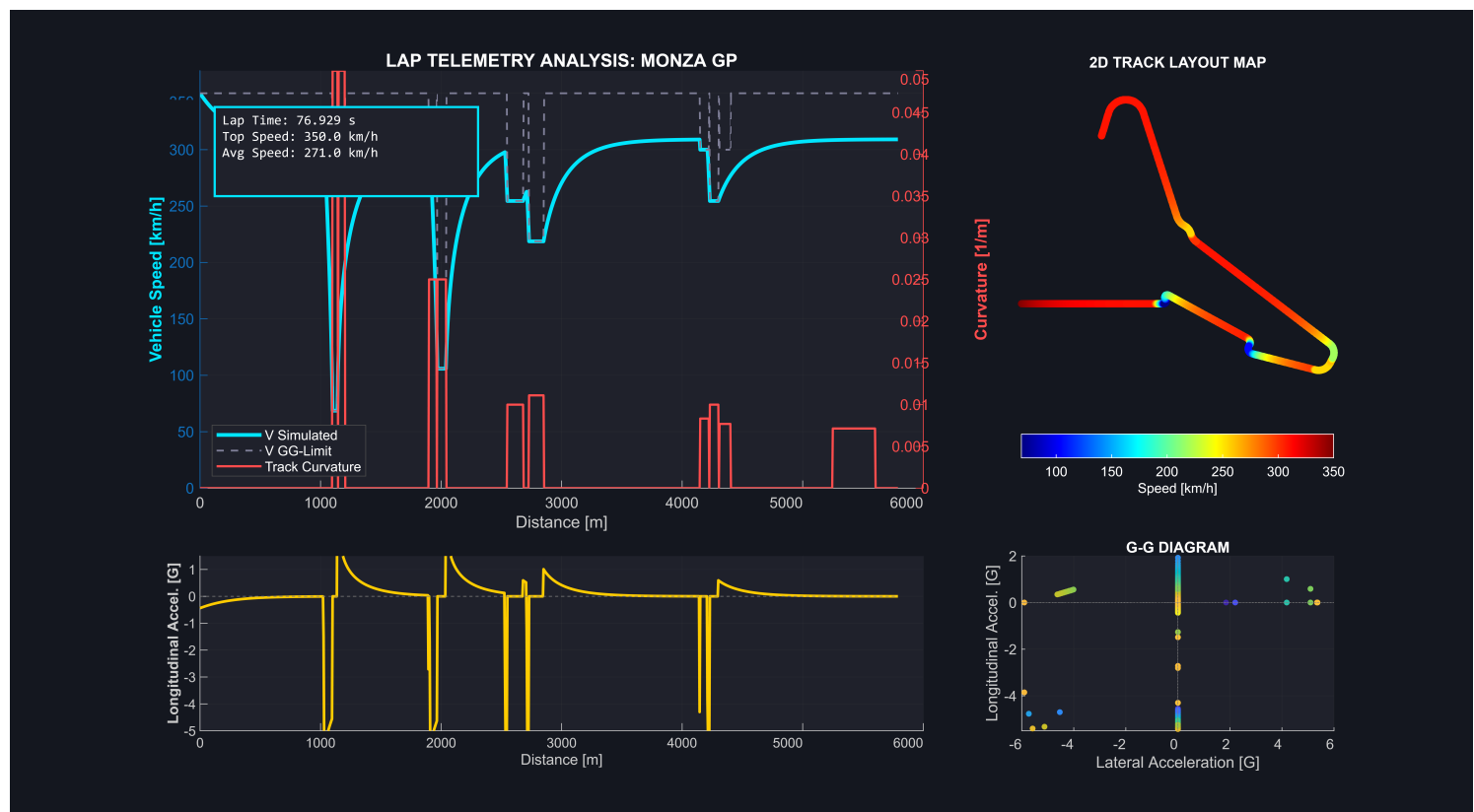
plot(ax4, [-6 6], [0 0], 'Color', [0.5 0.5 0.5], 'LineStyle', ':');
plot(ax4, [0 0], [-6 2], 'Color', [0.5 0.5 0.5], 'LineStyle', ':');

xlabel('Lateral Acceleration [G]');
ylabel('Longitudinal Accel. [G]');
title('G-G DIAGRAM', 'Color', 'w', 'FontSize', 10, 'FontWeight', 'bold');

xlim(ax4, [-6, 6]);
ylim(ax4, [-5.5, 2]);
axis(ax4, 'normal');

%% KPI Dashboard Overlay
annotation('textbox', [0.14, 0.77, 0.18, 0.11], 'String', ...
    {'Lap Time: ' num2str(lap_time, '%.3f') ' s'}, ...
    ['Top Speed: ' num2str(max(v_simulated) * 3.6, '%.1f') ' km/h'], ...
    ['Avg Speed: ' num2str((distance(end)/lap_time)*3.6, '%.1f') ' km/h']], ...
    'Color', 'w', 'EdgeColor', [0.0 0.9 1.0], 'LineWidth', 1.5, ...
    'BackgroundColor', [0.12 0.13 0.17 0.9], 'FontName', 'Consolas',
    'FontSize', 9);

```



5. Model limitations & future technical iterations

While this quasi-static point-mass model provides an excellent structural baseline for lap time estimation, a competitive F1 vehicle dynamics program requires accounting for higher-order complexities. Future revisions of this framework will introduce:

1. **Combined tire slip (friction ellipse):** replacing the abrupt minimum envelope approach with a continuous friction ellipse constraint, allowing the car to accelerate or brake *while* cornering using a realistic combined slip model.
2. **Transient load transfers:** implementing a 3-DOF or 7-DOF vehicle chassis model to simulate lateral and longitudinal load transfers, which directly affect individual tire vertical loads (F_z).
3. **Non-linear tire sensitivity:** transitioning from a constant friction coefficient (μ) to a non-linear tire model (e.g., Pacejka Magic Formula) that captures tire load sensitivity and slip angle variations.